

CSE222 / BİL505
Data Structures and Algorithms
Homework #6 – Report

Dicle Çoban

1) Selection Sort

Time Analysis	Worst case: $O(n^2)$ comparison, $O(n)$ swaps. Which means the time complexity of the algorithm grows quadratically as the number of elements increase. Best case: $O(n^2)$ comparison, $O(n)$ swaps. Even though its best case its still not efficient. Average Case: $O(n^2)$ comparison, $O(n)$ swaps
Space Analysis	It's in-place sorting so its only $O(1)$

2) Bubble Sort

Time Analysis	Worst case: $O(n^2)$ comparison, $O(n^2)$ swaps. This scenario occurs when the array is reverse ordered. Best case: $O(n)$ comparison, $O(1)$ swaps (already sorted) Average Case: $O(n^2)$ comparison, $O(n^2)$ swaps. It happens when the array is randomly shuffled.
Space Analysis	Just like Selection sort its in-place sorting so its only $O(1)$

3) Quick Sort

Time Analysis	Worst case: $O(n^2)$ comparison, $O(n^2)$ swaps. This happens when the array is reversed ordered. Best case: $O(n \log n)$ comparison, $O(\log n)$ swaps. This happens when the array is already sorted or nearly sorted. Average Case: $O(n \log n)$ comparison, $O(\log n)$ swaps. This happens when the array randomly shuffled.
----------------------	--

Space Analysis	It uses $O(\log n)$ for recursive part on best and average cases. For the worst case it uses $O(n)$ space
-----------------------	--

4) Merge Sort

Time Analysis	Worst case: $O(n \log n)$ comparison, $O(n \log n)$ swaps. This happens when the array is already sorted or reverse ordered. Best case: $O(n \log n)$ comparison, $O(n \log n)$ swaps. This happens when array is randomly shuffled. Average Case: $O(n \log n)$ comparison, $O(n \log n)$ swaps. This happens when array is randomly shuffled.
Space Analysis	It uses $O(n)$ space. If its recursively : $O(\log n)$ So the total is: $O(n + \log n) = O(n)$

General Comparison of the Algorithms

According to the informations that I fill above,

Quick Sort and Merge Sort are suitable for the large dataset due to their average case time complexity which is $O(n \log n)$. But Quick Sort something making some issues with the worst case time complexity cases and using additional space if it's not implemented carefully. We should pay attention to that.

If I compare Quick Sort and Merge Sort: Quick Sort can be faster due to lower constant factors and better cache performance. Merge Sort is stable and it would be a reliable choice for sorting large datasets.

Bubble Sort and Selection sort are less efficient comparing to the Quick Sort and Merge Sort due to large datasets.

If I compare Bubble Sort and Selection sort: They have similar time and space complexity, but I can say that Bubble sort has slightly worse due to number of swaps. But they are inefficient for the large datasets.